

Relatório Técnico – Projeto de Processador Customizado de 16-bits

1. Objetivo do Projeto

O objetivo deste projeto foi desenvolver, simular e validar um processador customizado de 16-bits utilizando o software Logisim. O desenvolvimento contemplou desde a implementação dos componentes fundamentais da arquitetura digital até a execução de aplicações complexas em linguagem Assembly.

O sistema foi projetado para suportar:

- Operações aritméticas e lógicas;
- Controle de fluxo por desvios condicionais e incondicionais;
- Manipulação de memória RAM;
- Execução de sub-rotinas utilizando pilha;
- Renderização gráfica em matriz de pixels;
- Interface textual via TTY;
- Execução de programas completos, incluindo um sistema operacional simplificado e um jogo estilo Tetris.

O projeto teve como foco principal aplicar conceitos de Arquitetura e Organização de Computadores, Sistemas Digitais e Programação de Baixo Nível.

2. Arquitetura Geral do Processador

A arquitetura completa do circuito foi organizada em módulos independentes interligados por um barramento principal de 16-bits. O projeto utiliza uma separação clara entre datapath, unidade de controle, memória e periféricos.

A CPU foi implementada utilizando arquitetura multiciclo com sinais de controle dedicados para leitura, escrita, movimentação de barramento e sincronização entre os módulos. A lógica de controle central coordena todos os módulos através de ROMs de microcódigo e sinais sincronizados pelo clock principal.

Os principais sinais identificados no circuito incluem:

- ACC_WRITE / ACC_READ
- RAM_IN / RAM_OUT
- MAR_READ / MAR_WRITE
- INSTRUCT_READ / INSTRUCT_WRITE
- STACK_UP / STACK_DOWN
- UPDATE_DISPLAY
- WRITE_X / WRITE_Y
- READ_X / READ_Y

- JUMP
- HALT

2.1 Fluxo de Execução do Sistema

1. O Program Counter (PC) fornece o endereço da próxima instrução.
2. A memória ROM retorna a instrução correspondente.
3. A instrução é armazenada no Instruction Register (IR).
4. A Unidade de Controle interpreta o opcode.
5. O Step Counter coordena os micro-passos da execução.
6. Os dados trafegam pelo barramento principal de 16-bits.
7. A ULA executa operações aritméticas e lógicas.
8. Os resultados são armazenados em registradores ou memória.

2.2 Principais Componentes do Datapath

- Program Counter (PC)
- Instruction Register (IR)
- Unidade Lógica e Aritmética (ULA)
- Banco de Registradores
- Unidade de Controle e Step Counter (FSM multíciclo)
- Memória RAM e Video RAM (VRAM)
- Frame Buffer e Interface TTY
- Registrador de Flags

3. Organização do Conjunto de Instruções (ISA)

As instruções possuem largura fixa de 16-bits, divididas conforme a tabela abaixo:

Bits	Função
15 – 8	Opcode (Definição da operação pela Unidade de Controle)
7 – 0	Operando imediato ou endereço de memória

4. Descrição dos Principais Módulos

4.1 Unidade Lógica e Aritmética (ULA)

A ULA foi implementada como um módulo independente de 16-bits contendo entradas A e B, sinais de operação dedicados e geração automática de flags. O circuito possui barramentos de entrada e saída de 16-bits, além de pinos de Carry

In, Borrow In, Carry Out e Borrow Out. As operações são selecionadas diretamente através de linhas de controle individuais fornecidas pela Unidade de Controle.

As operações implementadas estão divididas em:

- **Operações Aritméticas:** ADD, SUB, ADI (soma imediata) e SUI (subtração imediata).
- **Operações Lógicas:** NOT, AND, OR, XOR, NAND, NOR.

4.1.1 Registrador de Flags

As operações executadas pela ULA atualizam automaticamente o registrador de flags de acordo com as condições do resultado:

Flag	Função
ZERO	Ativada quando o resultado é igual a zero
GREATER	Ativada quando o operando A é maior que B
LESS	Ativada quando o operando A é menor que B
EQUALS	Ativada quando os valores de A e B são equivalentes
CARRY	Ativada em caso de overflow em operações de soma
BORROW	Ativada em caso de borrow em operações de subtração

4.2 Banco de Registradores

O processador possui múltiplos registradores de 16-bits para armazenamento temporário e manipulação de ponteiros:

Registrador	Função
ACC	Acumulador principal (atua como entrada e saída obrigatória da ULA)
B	Registrador auxiliar para operações gerais
X	Ponteiro de memória (essencial para manipulação gráfica e indexação)

Y	Ponteiro auxiliar (utilizado em conjunto com o registrador X)
OUT	Interface de saída do processador

4.3 Unidade de Controle

A Unidade de Controle opera utilizando uma arquitetura multíciclo baseada em uma Máquina de Estados Finitos (FSM), implementada por meio de microcódigo armazenado em ROMs dedicadas. Esse modelo reduz problemas relacionados ao tempo de propagação do circuito, melhora a estabilidade geral da arquitetura e isola o hardware lógico da lógica de controle.

O sistema utiliza um **Step Counter de 4-bits** para definir o estágio atual da execução de cada instrução. Por possuir 4-bits, o sistema suporta até 16 micro-steps por instrução, divididos nas etapas clássicas de: Fetch, Decode, Execute e Write Back.

Os sinais produzidos pelas ROMs de microcódigo controlam diretamente a escrita/leitura de registradores, barramento, operações da ULA, acesso à RAM, controle de pilha, renderização gráfica e o fluxo do Program Counter.

4.4 Memória e Subsistema de Vídeo

O hardware gerencia de forma independente a memória principal do sistema e o espaço de endereçamento de vídeo (VRAM):

Memória	Organização
ROM Principal	64K x 16
RAM Principal	64K x 16
RAM Auxiliar	256 x 16

O subsistema gráfico foi implementado utilizando uma arquitetura baseada em Frame Buffer e Video Buffer. A renderização ocorre através da escrita lógica no Frame Buffer com base em coordenadas X/Y e controle por endereço, seguida da atualização física do display matricial. O sistema também possui saída textual baseada em uma interface TTY de 12 linhas por 32 colunas e um buffer de teclado com capacidade para 32 caracteres.

5. Instruções Implementadas

5.1 Load/Store

Hex	Mnemônico	Descrição
01XX	LDA	Carrega valor da RAM no ACC
02XX	LDB	Carrega valor da RAM no registrador B
0BXX	STA	Armazena o valor do ACC na RAM
0CXX	STB	Armazena o valor do registrador B na RAM
0FXX	STK	Armazena o valor do registrador do teclado na RAM

5.2 Imediatos

Hex	Mnemônico	Descrição
07XX	LIB	Carrega um valor imediato no registrador B
13XX	ADI	Soma um valor imediato ao ACC
14XX	SUI	Subtrai um valor imediato do ACC

5.3 Operações da ULA

Hex	Mnemônico	Descrição
10XX	ADD	Soma o valor contido na RAM ao ACC
11XX	SUB	Subtrai o valor contido na RAM do ACC
12XX	CMP	Compara o valor do ACC com um valor da RAM
50XX	LNR	Executa o NOT lógico do ACC e salva na RAM

5.4 Controle de Fluxo

Hex	Mnemônico	Descrição
70XX	JMP	Realiza um salto incondicional para o endereço especificado
72XX	JPZ	Realiza um salto condicional se a flag ZERO estiver ativa
74XX	JPG	Realiza um salto condicional se a flag GREATER estiver ativa

5.5 Pilha e Sub-rotinas

Hex	Mnemônico	Descrição
32XX	CAI	Realiza o Push do PC na stack (pilha) e desvia o fluxo (salto)
20XX	SYX	Armazena o valor de Y no endereço de memória apontado por X

6. Exemplo de Execução Multiciclo

Para demonstrar a separação física entre as etapas de busca, decodificação e execução da máquina de estados, o fluxo da instrução LDA 0x20 ocorre conforme os passos cronológicos descritos abaixo:

Nº	Operação
1	O valor do Program Counter é enviado ao Memory Address Register (PC → MAR)
2	A instrução é lida da ROM e armazenada no Instruction Register (ROM → IR)
3	O operando de endereço é isolado e enviado ao registrador de endereço (Operando → MAR)

4	O dado contido no endereço da RAM é carregado no Acumulador (RAM → ACC)
---	---

7. Programas Desenvolvidos

7.1 Sistema Operacional Simplificado

Foi desenvolvido um sistema operacional em Assembly capaz de:

- Receber entrada via teclado;
- Exibir interface textual em TTY;
- Gerenciar rotinas de memória;
- Salvar instruções digitadas pelo usuário;
- Executar código dinamicamente.

O sistema utiliza ativamente sub-rotinas, pilha de chamadas e rotinas estruturadas de limpeza de memória (CLEAR_RAM).

7.2 Jogo Tetris

Um clone completo do jogo Tetris foi desenvolvido para rodar sobre o hardware customizado. O software implementa:

- Manipulação de matrizes bidimensionais para representação das peças;
- Renderização controlada por Frame Buffer;
- Rotinas lógicas para controle de colisões e queda;
- Atualização dinâmica da VRAM;
- Controle de movimento em tempo real mapeado por entradas do teclado.

As peças são desenhadas e deslocadas na matriz gráfica utilizando ponteiros de memória e operações lógicas envolvendo diretamente os registradores X e Y.

8. Resultados Obtidos

Durante a fase de testes, o sistema demonstrou funcionamento estável e integração funcional completa entre todos os módulos lógicos do processador. O modelo multiciclo baseado na FSM eliminou falhas críticas de temporização e de sincronização no barramento compartilhado.

A arquitetura validou com sucesso conceitos avançados de engenharia de computação, tais como o controle microprogramado, gerenciamento de sub-rotinas via pilha de hardware e o mapeamento de periféricos gráficos em memória isolada

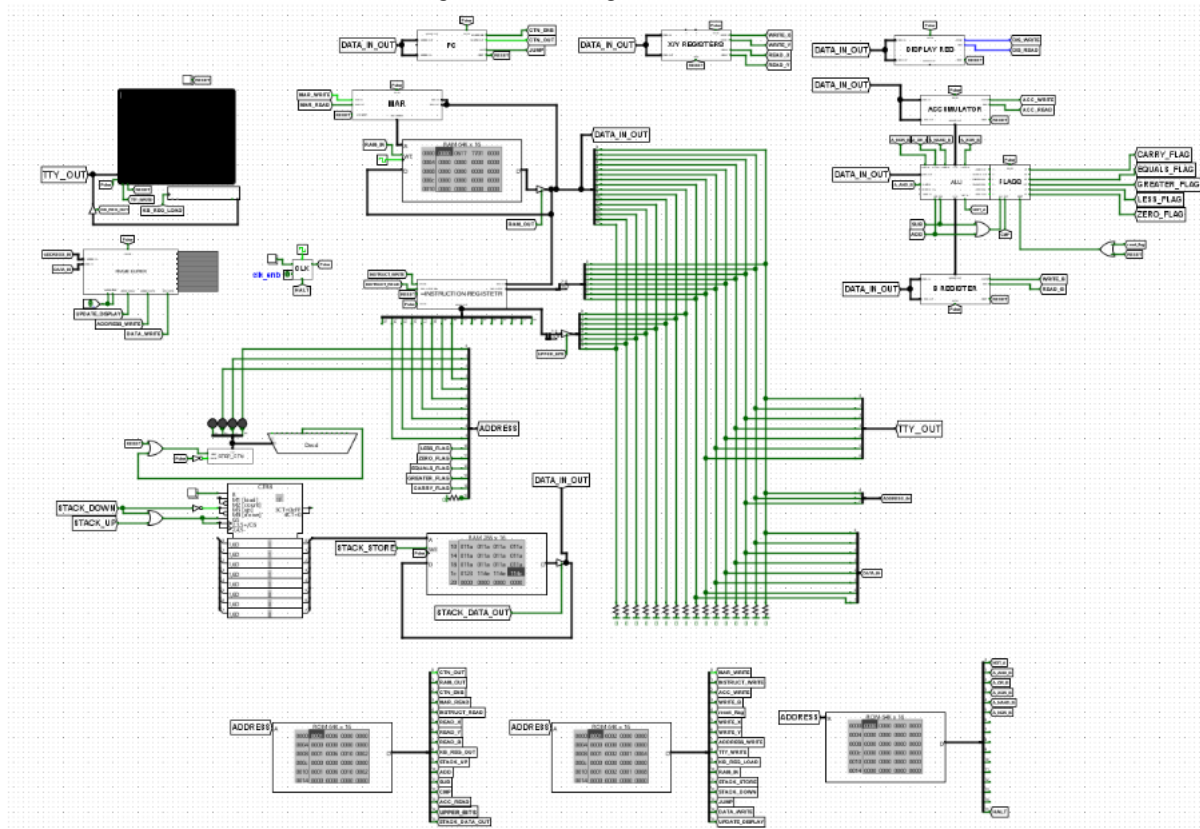


Figura 2: Funcionamento e barramentos:

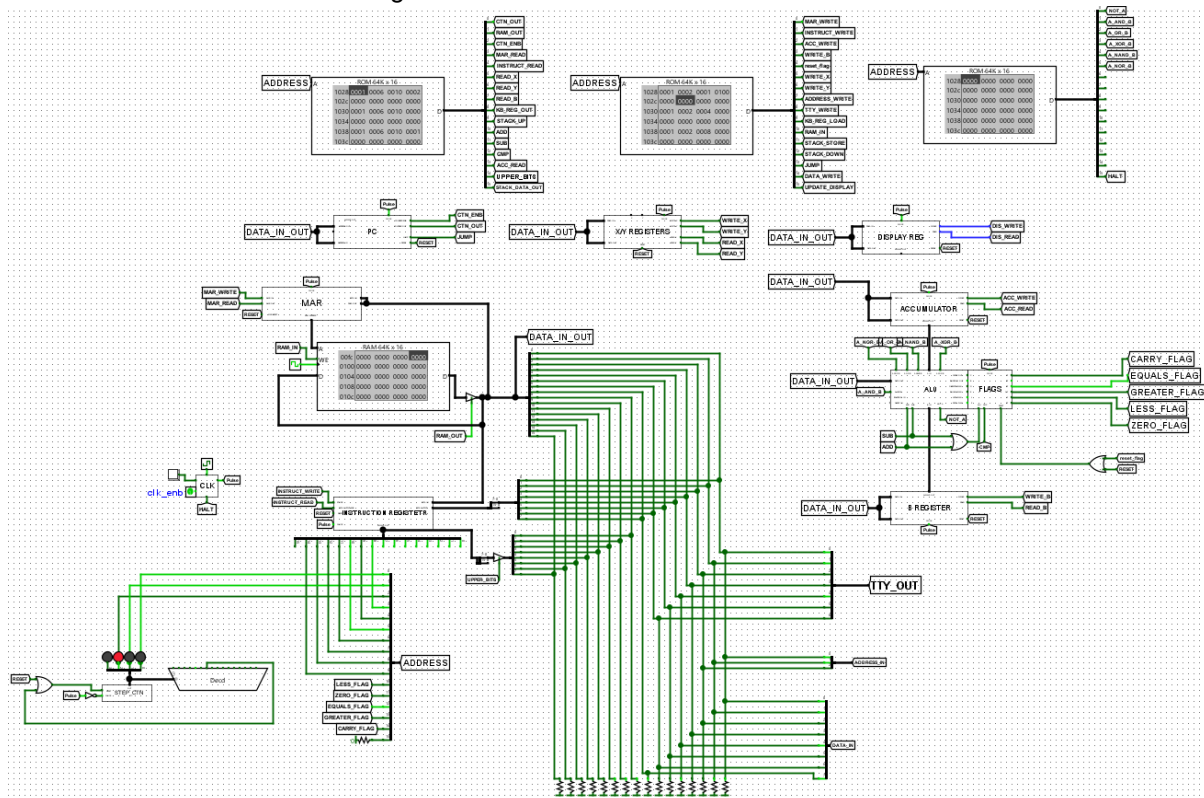
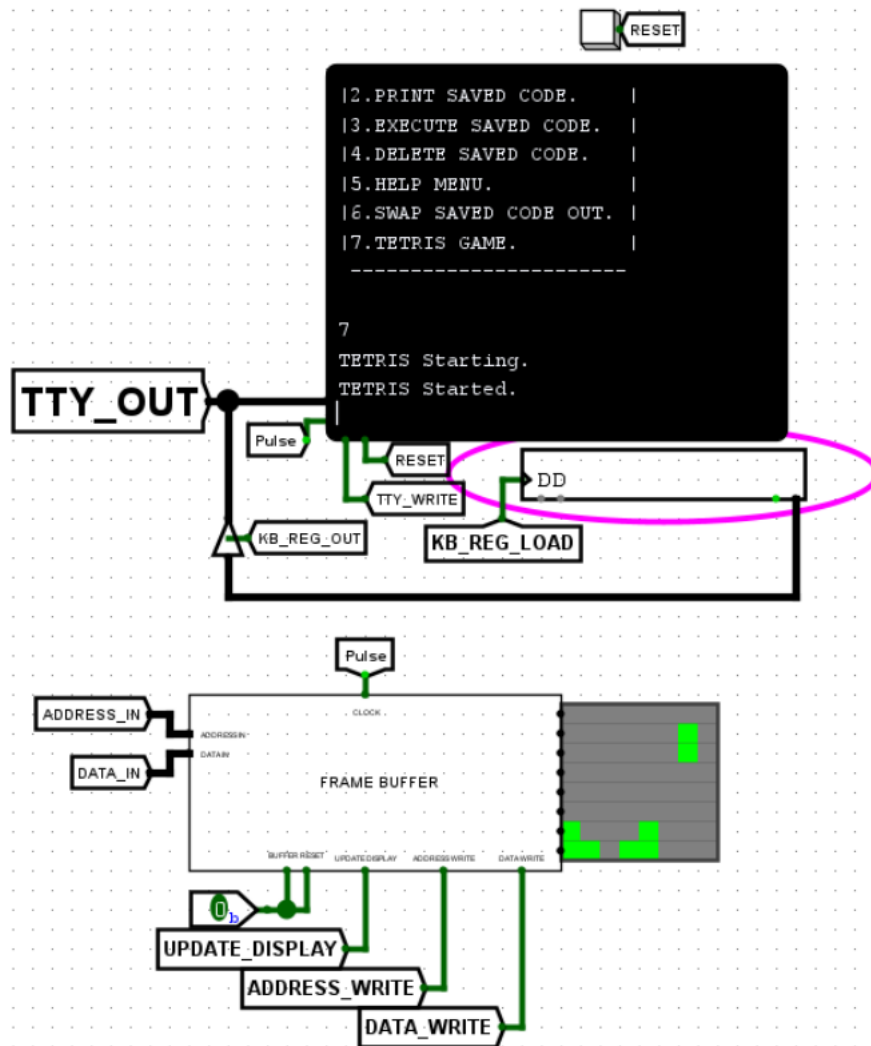


Figura 3: Sistema operacional TTY

```
|  CUSTOM ASSEMBLY O.S.  |
|ENTER SELECTION BELOW. |
|1.INPUT ASSEMBLY CODE. |
|2.PRINT SAVED CODE.    |
|3.EXECUTE SAVED CODE.  |
|4.DELETE SAVED CODE.   |
|5.HELP MENU.           |
|6.SWAP SAVED CODE OUT. |
|7.TETRIS GAME.         |
|-----|
|
```

Figura 4: Tetris em execução



10. Possíveis Melhorias

10.1 Sistema de Interrupções

Substituir a lógica atual baseada em *polling* por interrupções de hardware dedicadas na leitura do teclado, evitando loops de espera bloqueantes e otimizando o uso dos ciclos de clock.

10.2 Pipeline

Implementar uma arquitetura baseada em estágios de *pipelining* para permitir a execução paralela de instruções, elevando o rendimento geral da CPU e o IPC (*Instructions Per Cycle*).

10.3 Multiplicação e Divisão em Hardware

Desenvolver blocos aritméticos dedicados na ULA para multiplicação e divisão, eliminando a necessidade de rotinas complexas em software que elevam a latência computacional.

10.4 Expansão do ISA

Adicionar suporte a registradores de deslocamento dedicados (*shift registers*), modos de endereçamento indireto avançado e expansão do banco de registradores gerais.

11. Conclusão

O projeto atingiu plenamente os objetivos propostos ao implementar uma arquitetura funcional de processador de 16-bits a partir de blocos lógicos fundamentais no Logisim. A validação prática do hardware por meio de software de baixo nível complexo (SO e Tetris) comprovou a eficiência e o correto funcionamento do design multiciclo, consolidando conceitos de organização de memória, sistemas embarcados e o acoplamento correto entre hardware e software.